IMAGE COMPRESSION TOOL USING HUFFMAN CODING AND TRANSFORM-BASED BLOCK PROCESSING

PROJECT REPORT
Submitted in partial fulfilment of the requirements
for the award of degree of

MASTER OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

by

RAMA SYAM SUNDAR 2601050102

Under the supervision of

Dr Raju Anitha
Professor



Department of Computer Science and Engineering
Koneru Lakshmaiah Education Foundation (Deemed
to be University estd., u/s 3 of UGC Act 1956)
Green Fields, Vaddeswaram-522502, Guntur (Dist.), Andhra Pradesh, India

KONERU LAKSHMAIAH EDUCATION FOUNDATION
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



DECLARATION

The Project Report entitled "Image Compression Tool Using Huffman Coding and Transform-Based Block Processing" is a record of my own bona fide work submitted in partial fulfilment of the requirements for the award of the Master of Technology degree in Computer Science and Engineering to K L Deemed to be University during the academic year 2026–2027.

RAMA SYAM SUNDAR
Student ID: 2601050102

KONERU LAKSHMAIAH EDUCATION FOUNDATION
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is to certify that the report entitled "Image Compression Tool Using Huffman Coding and Transform-Based Block Processing" is being submitted by RAMA SYAM SUNDAR (Student ID: 2601050102) in partial fulfilment of the requirements for the award of the Master of Technology degree in Computer Science and Engineering at K L Deemed to be University during the academic year 2026–2027.

Dr Raju Anitha
Professor

ACKNOWLEDGEMENT

I express my sincere gratitude to my project guide, Dr Raju Anitha, for the valuable guidance, constant encouragement and constructive suggestions that helped me to complete this project successfully.

I thank the Head of the Department of Computer Science and Engineering and the Principal of K L Deemed to be University for providing the facilities and a supportive environment for the work.

I am also thankful to all the faculty members of the department, to the open-source communities whose tools made this project possible, and to my family and friends for their continuous support.

Rama Syam Sundar

Abstract

Digital images dominate storage and network traffic, and efficient compression remains a core problem in multimedia systems. This project presents the design, implementation and evaluation of a complete image compression and decompression tool built from two classical ideas: transform-based block processing and Huffman coding. The input image is converted to grayscale and divided into independent 8×8 blocks (divide and conquer). Each block is mapped to the frequency domain with the two-dimensional Discrete Cosine Transform (DCT), quantised with a quality-controlled table, and serialised into a symbol stream using differential DC coding, zig-zag scanning and end-of-block markers. A Huffman tree is then built with a min-heap (a greedy algorithm), codes are generated and the stream is bit-packed into a self-describing binary file. The decompressor reverses each step: Huffman decoding, reconstruction of quantised blocks, dequantisation, inverse DCT and block merging.

The system is implemented in Python 3.11+ with an object-oriented structure of eight core classes, a command-line interface, unit tests and complete documentation. It was evaluated on a synthetic test image and two natural photographs at five quality settings. At quality 50 the tool achieved compression ratios of 26.6:1, 10.8:1 and 6.9:1 relative to raw 8-bit grayscale, with PSNR between 28.2 dB and 38.8 dB. Applying Huffman coding directly to pixels, without the transform, gave only 1.03:1 to 1.12:1, confirming that the transform and quantisation stages are what make the data compressible. Compared with the OpenCV JPEG encoder using optimised Huffman tables, the produced files were 4% to 25% larger at essentially identical PSNR, mainly because the symbol model has no joint (run, size) symbols. The measured running time grows linearly with image size, in agreement with the complexity analysis.

Keywords: image compression, Huffman coding, discrete cosine transform, block transform coding, quantisation, greedy algorithm, divide and conquer.

Table of Contents

1. Introduction..... 3
2. Problem Statement..... 4
3. Literature Survey..... 4
4. Methodology..... 6
5. System Design..... 7
6. Algorithms..... 9
7. Implementation..... 11
8. Experimental Results..... 12
9. Complexity Analysis..... 16
10. Advantages..... 17
11. Limitations..... 17
12. Future Scope..... 18
13. Conclusion..... 18
References..... 18

1. Introduction

1.1 Background

A grayscale image of 512×512 pixels occupies 262,144 bytes when stored as raw 8-bit samples, and a modest photograph from a mobile phone occupies tens of megabytes in uncompressed form. Storing and transmitting such data without compression is wasteful because images contain large amounts of redundancy. Neighbouring pixels are strongly correlated (spatial redundancy), the distribution of sample values is far from uniform (coding redundancy), and the human visual system is insensitive to some high-frequency detail (psychovisual redundancy) [Gonzalez and Woods, 2018]. Image compression exploits these three kinds of redundancy to reduce the number of bits needed to represent an image.

Practical lossy compressors, of which JPEG is the best-known example, follow a common pattern: decorrelate the data with a transform, discard perceptually unimportant information by quantisation, and entropy-code what remains [Wallace, 1992]. This project implements that pattern end to end in a compact, readable code base and uses it as a vehicle to study two algorithm design paradigms: the greedy method (Huffman coding) and divide and conquer (independent block processing).

1.2 Motivation

Compression algorithms are an excellent teaching and research subject because they combine mathematics (transform theory, information theory), data structures (heaps, binary trees, bit streams) and software engineering (file formats, error handling, evaluation). A from-scratch implementation makes every design decision explicit and measurable, which is not possible when a library such as libjpeg is used as a black box. The project also provides a baseline against which improvements such as run-length symbols, canonical codes or arithmetic coding can be assessed.

1.3 Objectives

- Design a compression workflow that divides an image into 8×8 blocks, applies the DCT to each block and Huffman-codes the transformed data into a `.bin` file.
- Design the inverse workflow that reconstructs the image from the `.bin` file.
- Implement the system in an object-oriented, modular and PEP 8 style code base with error handling and tests.
- Evaluate compression ratio, reconstruction quality (MSE, PSNR) and running time, and compare the design with plain Huffman coding and with a reference JPEG encoder.
- Document the design, algorithms and complexity in a form suitable for a public repository.

1.4 Scope

The tool handles single images of arbitrary size, converted to 8-bit grayscale. Colour handling, chroma subsampling, progressive coding, video and standards-compliant JPEG output are outside the scope of the project. The compressor is lossy because of quantisation; the quality parameter selects the trade-off between size and fidelity, and quality 100 gives near-lossless reconstruction.

1.5 Organisation of the Report

Chapter 2 defines the problem. Chapter 3 surveys the relevant literature. Chapter 4 describes the methodology and Chapter 5 the system design. Chapter 6 explains the algorithms and Chapter 7 the implementation. Chapter 8 reports the experimental results, Chapter 9 gives the complexity analysis, and Chapters 10 to 13 discuss advantages, limitations, future scope and conclusions.

2. Problem Statement

Given an 8-bit image of arbitrary dimensions, **design and implement** a system that (a) produces a compressed binary file substantially smaller than the raw pixel data and (b) reconstructs an image from that file whose distortion with respect to the original is controllable and quantifiable.

The solution must satisfy the following requirements:

1. **Divide and conquer:** the image must be partitioned into 8×8 blocks that are processed independently and merged after processing.
2. **Transform coding:** each block must be transformed with the DCT, and the inverse DCT must be used for reconstruction.
3. **Greedy entropy coding:** the transformed data must be encoded with Huffman coding using a priority queue; frequency calculation, tree construction, code generation, encoding and decoding must all be implemented.
4. **Object-oriented design:** the functionality must be organised into the classes ImageProcessor, TransformBlock, HuffmanNode, HuffmanTree, Encoder, Decoder, CompressionManager and DecompressionManager.
5. **Usability and quality:** the tool must run from the command line, validate its inputs, and be accompanied by documentation, tests and a complexity analysis.

A subtle issue shapes the solution. The DCT of 8-bit data yields real-valued coefficients, and Huffman coding of real numbers with (nearly) all distinct values gives no compression. A quantisation step that maps coefficients to a small set of integers is therefore a necessary part of the pipeline even though it is not listed explicitly in the problem workflow. This step is the only source of information loss.

3. Literature Survey

3.1 Entropy and Huffman Coding

Shannon showed that the average number of bits per symbol needed to represent a source cannot fall below its entropy [Shannon, 1948]. Huffman's 1952 algorithm constructs a prefix code that is optimal among all codes that assign an integer number of bits to each symbol, with an average length L satisfying $H \leq L < H + 1$ [Huffman, 1952]. The algorithm is greedy: it repeatedly merges the two least probable symbols, and a standard exchange argument proves that this local choice leads to a globally optimal code [Cormen et al., 2022]. With a binary heap the construction takes $O(n \log n)$ time for n distinct symbols. Gallager analysed the redundancy of Huffman codes and the properties of the resulting trees [Gallager, 1978], and Schwartz and Kallick introduced canonical codes, which allow a decoder to reconstruct the code from code lengths alone [Schwartz and Kallick, 1964].

3.2 Arithmetic Coding

Arithmetic coding represents an entire message as a sub-interval of [0, 1) and can approach the entropy to within a tiny overhead, even for symbols whose probabilities are not powers of two [Witten et al., 1987]. It also adapts naturally to changing statistics. Its cost is higher implementation complexity, and historically patent concerns slowed its adoption in image standards. Context-adaptive binary arithmetic coding was later adopted in video standards such as H.264/AVC [Wiegand et al., 2003]. Huffman coding remains attractive for educational and lightweight implementations because of its simplicity and speed.

3.3 Transform Coding and the Discrete Cosine Transform

Transform coding maps a block of correlated samples into a set of less correlated coefficients so that most of the signal energy is concentrated in a few of them. The optimal linear transform for a Gaussian source is the Karhunen–Loève transform, but it depends on the signal statistics and has no fast algorithm. Ahmed, Natarajan and Rao introduced the DCT in 1974 and showed that its performance on first-order Markov data with high correlation closely approaches that of the Karhunen–Loève transform [Ahmed et al., 1974]. The DCT is real-valued, separable, has an orthonormal form and admits fast algorithms such as that of Loeffler, Ligtenberg and Moschytz, which computes the 1-D 8-point DCT with 11 multiplications [Loeffler et al., 1989]. The comprehensive treatment of Rao and Yip covers the theory, algorithms and applications [Rao and Yip, 1990]. Compared with the discrete Fourier transform, the DCT implicitly uses a symmetric extension of the block, which avoids the boundary discontinuities responsible for spectral leakage and makes it better suited to compression.

3.4 The JPEG Standard

The baseline JPEG standard divides an image into 8×8 blocks, applies the DCT, quantises the coefficients with a perceptually tuned table, orders them in zig-zag fashion, codes the DC coefficients differentially, run-length codes zero runs in the AC coefficients and entropy-codes the resulting symbols with Huffman tables [Wallace, 1992; Pennebaker and Mitchell, 1993]. The quantisation tables are scaled by a quality factor, which is the same mechanism adopted in this project. The main difference between JPEG and this implementation lies in the symbol model: JPEG forms joint (zero-run, magnitude-category) symbols followed by raw magnitude bits, whereas this tool Huffman-codes the coefficient values themselves and relies on end-of-block markers to remove trailing zeros.

3.5 Wavelet-Based Compression

The JPEG 2000 standard replaces the block DCT by a wavelet transform applied to the whole image or to large tiles, followed by embedded block coding with optimised truncation and arithmetic coding [Taubman and Marcellin, 2002; Skodras et al., 2001]. It avoids the blocking artefacts typical of low-bit-rate DCT coding and supports scalability, at the cost of considerably higher complexity. For a project that must demonstrate block-wise divide and conquer, the block DCT is the natural choice.

3.6 Image Quality Measurement

Mean squared error (MSE) and peak signal-to-noise ratio (PSNR) are the most common objective measures of reconstruction fidelity. They are simple, differentiable and cheap, but they correlate only loosely with human perception. Structural measures such as SSIM compare

local luminance, contrast and structure [Wang et al., 2004]. This project reports MSE and PSNR; SSIM is left as an extension.

3.7 Summary and Positioning

Table 1: Comparison of related techniques

Technique	Core idea	Strength	Weakness
Huffman coding	Optimal integer-length prefix code	Simple, fast, optimal for its class	Cannot exploit pixel correlation alone
Arithmetic coding	Interval subdivision	Near-entropy rate, adaptive	More complex to implement
Block DCT (JPEG)	Decorrelate 8×8 blocks	Energy compaction, fast	Blocking artefacts at low rates
Wavelet (JPEG 2000)	Multi-resolution transform	No blocking, scalable	Higher complexity
This project	Block DCT + quantisation + Huffman	Transparent, modular, measurable	No run-length symbols, grayscale only

The literature shows that the combination of a block DCT, quantisation and Huffman coding is a well-understood and effective architecture. The contribution of this project is not a new algorithm but a clean, tested and documented implementation that exposes each stage, a quantified analysis of how much each modelling decision contributes to the compression ratio, and a comparison against a production JPEG encoder.

4. Methodology

4.1 Overall Approach

The work followed an iterative engineering methodology: (1) analyse the requirements and fix the pipeline; (2) design the class structure and file format; (3) implement and unit-test each component separately, starting with the leaf components (HuffmanNode, HuffmanTree, TransformBlock) and finishing with the managers; (4) integrate the pipeline behind a command-line interface; (5) evaluate on test images and refine the design; (6) document the system. Reversibility was verified at each stage: the DCT round trip is exact up to rounding, the Huffman round trip is exactly lossless, and only quantisation introduces error.

4.2 Design Principles

- **Separation of concerns:** pixel-domain operations, transform operations, entropy coding and orchestration live in separate classes.
- **Determinism:** Huffman tree construction breaks ties by insertion order so that the decoder can rebuild the identical tree from the stored frequency table.
- **Self-describing output:** the `.bin` file carries a magic number, version, image size, quality, and frequency table, so decompression needs no side information.
- **Fail loudly:** all malformed inputs raise explicit errors instead of producing wrong images.
- **Measurability:** every stage exposes statistics so that its contribution can be evaluated.

4.3 Compression and Decompression Workflow

Figure 1 shows the two pipelines. Compression proceeds from the input image to the `.bin` file through ten steps, and decompression retraces the steps in reverse order.

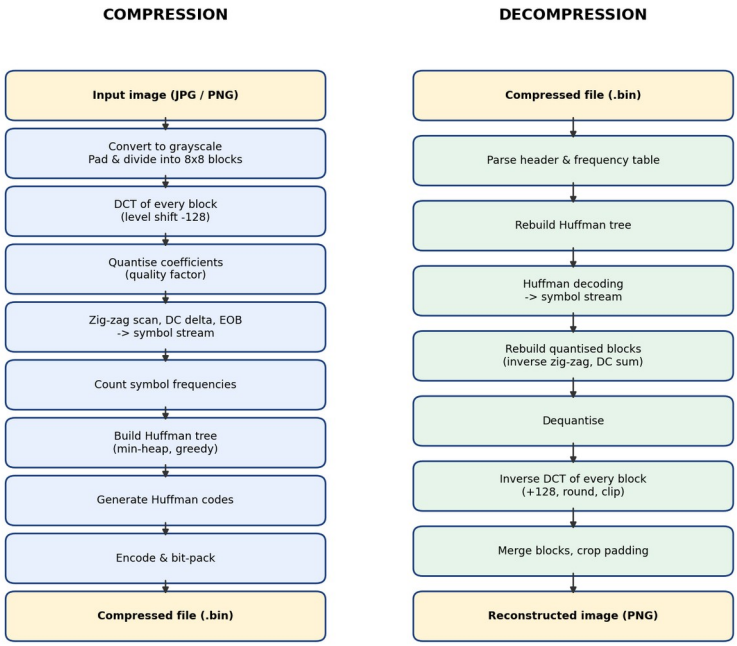


Figure 1: Compression and decompression workflow

4.4 Evaluation Methodology

The tool was evaluated with three test images: a synthetic 512×512 picture (smooth gradients, a disc, a rectangle, a sharp line, a noisy texture patch and text), a 512×600 portrait photograph (Photo 1, matplotlib sample data) and a 640×427 outdoor photograph (Photo 2, scikit-learn sample data). Each was compressed at quality factors 10, 25, 50, 75 and 90. The metrics were compressed file size, compression ratio relative to the raw 8-bit samples (H×W bytes), bits per pixel, MSE and PSNR between the original grayscale image and the reconstruction. Ablation experiments removed individual design elements (zig-zag with end-of-block markers, differential DC coding, the transform itself) to measure their contribution. Finally, the files were compared with the output of the OpenCV JPEG encoder at the same nominal quality, and running times were measured for increasing image sizes.

5. System Design

5.1 Architecture

The system has four layers (Figure 2). The presentation layer is a command-line program offering `compress`, `decompress` and `evaluate` commands. The orchestration layer consists of CompressionManager and DecompressionManager, which call the processing classes in the correct order and handle files. The processing layer contains ImageProcessor, TransformBlock, Encoder and Decoder. The lowest layer holds the core data structures, HuffmanTree and HuffmanNode, together with a utility module for the file container, the zig-zag order and the quality metrics. Dependencies point only downwards.

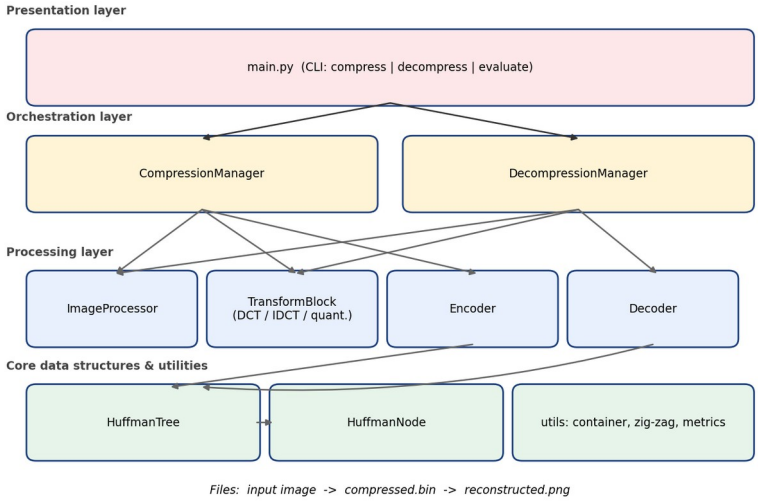


Figure 2: Layered architecture of the system

5.2 Class Design

Figure 3 shows the class diagram. Each class has one responsibility, as listed in Table 2.

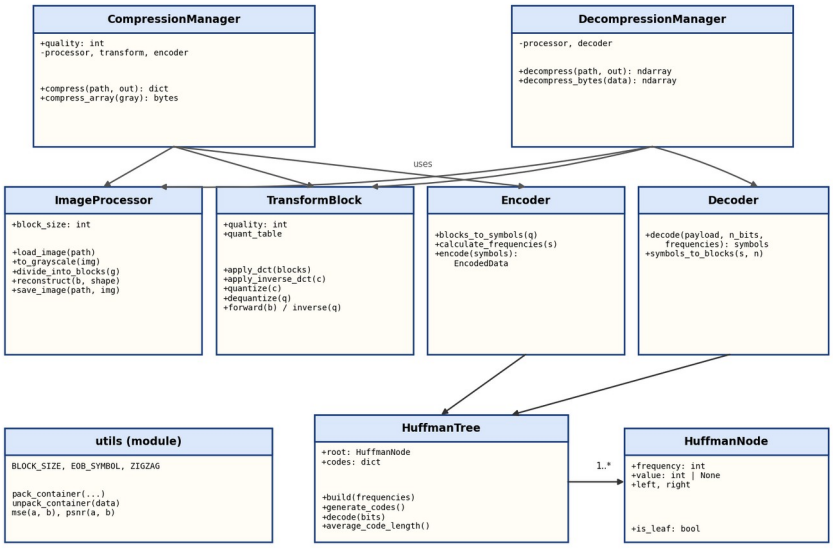


Figure 3: Class diagram

Table 2: Classes and responsibilities

Class	Responsibility
ImageProcessor	Loads images, converts to grayscale, pads and divides into 8×8 blocks, merges blocks and crops padding, saves images.
TransformBlock	Forward and inverse 2-D DCT; quantisation and dequantisation controlled by the quality factor.
HuffmanNode	Tree node with frequency, value, left child and right child.
HuffmanTree	Builds the tree with a min-heap, generates the codes, decodes bit sequences.
Encoder	Builds the symbol stream, counts frequencies, performs Huffman encoding and bit packing.
Decoder	Rebuilds the tree from frequencies, decodes bits and rebuilds quantised blocks.

Class	Responsibility
CompressionManager	Runs the compression pipeline and writes the `.bin` file with statistics.
DecompressionManager	Reads the `.bin` file, runs the decompression pipeline and writes the image.

5.3 Compressed File Format

The `.bin` container (Table 3) consists of a 26-byte header, a frequency table and the Huffman payload. Storing the frequency table rather than the codes keeps the decoder simple: it calls the same deterministic tree-building routine as the encoder. Metadata is serialised with Python's `struct` module rather than `pickle`, because unpickling data from an untrusted file can execute arbitrary code.

Table 3: Layout of the compressed file (little-endian)

Field	Size	Description
Magic	4 bytes	ASCII "HIMG", identifies the format
Version	1 byte	Container version (currently 1)
Height, Width	4 + 4 bytes	Original image dimensions in pixels
Quality	1 byte	Quality factor 1–100, selects the quantisation table
Symbol count	4 bytes	Number n of entries in the frequency table
Bit count	8 bytes	Number of valid bits in the payload
Frequency table	8·n bytes	n pairs of (int32 symbol, uint32 frequency)
Payload	variable	Huffman bit stream, MSB first, zero-padded to a byte

5.4 Error Handling

The loader rejects missing files and undecodable images. The container parser verifies the magic number, version, table length and payload length. The symbol-stream parser checks for missing end-of-block markers, blocks with more than 63 AC values, exhausted streams and trailing symbols. Each of these conditions raises an exception that the command-line program converts to a message and a non-zero exit status.

6. Algorithms

6.1 Image Partitioning (Divide and Conquer)

Let the image have H rows and W columns. The padded dimensions are $H' = 8 \cdot \lceil H/8 \rceil$ and $W' = 8 \cdot \lceil W/8 \rceil$; missing rows and columns are filled by replicating the nearest edge pixel, which keeps the padded blocks smooth and therefore cheap to code. The padded image is divided into $B = (H'/8) \cdot (W'/8)$ blocks in raster order. Divide: split into blocks. Conquer: transform and quantise each block independently. Combine: merge the reconstructed blocks in the same order and crop to H×W. Because blocks do not depend on each other, the conquer step is embarrassingly parallel; the implementation processes all blocks in a single vectorised call.

6.2 Discrete Cosine Transform

After subtracting 128 from each sample to centre the data at zero, a block f(x, y) is transformed by the orthonormal 2-D DCT-II:

$$F(u,v) = \tfrac{1}{4} \cdot C(u) \cdot C(v) \cdot \Sigma x\, \Sigma y\, f(x,y) \cdot \cos[(2x+1)u\pi/16] \cdot \cos[(2y+1)v\pi/16], \quad x, y, u, v = 0\ldots7$$

where C(k) = 1/√2 for k = 0 and 1 otherwise. F(0,0) is the DC coefficient, proportional to the block mean; the remaining 63 coefficients are AC coefficients representing increasing horizontal and vertical spatial frequencies. The transform is separable and can be written as $F = T \cdot f \cdot T^{T}$ with an 8×8 orthogonal matrix T, so it is computed as two 1-D transforms, along rows and then columns. The inverse transform is $f = T^{T} \cdot F \cdot T$. The implementation uses `scipy.fft.dctn` and `idctn` with `norm="ortho"`.

6.3 Quantisation

Each coefficient is divided by the corresponding entry of a quantisation table Q and rounded to the nearest integer: Fq(u,v) = round(F(u,v)/Q(u,v)). The table is the standard JPEG luminance table scaled by a factor s that depends on the quality q ∈ [1, 100]:

$$s = 5000/q \text{ for } q < 50, \quad s = 200 - 2q \text{ for } q \geq 50, \quad Q = \mathrm{clip}(\lfloor (Q50 \cdot s + 50)/100 \rfloor, 1, 255)$$

Large divisors at high frequencies map small coefficients to zero, which is where most of the compression is obtained and where the loss occurs. Dequantisation multiplies by Q. At q = 100 all divisors are 1, so only the rounding of coefficients remains.

6.4 Symbol Stream Construction

The quantised blocks are serialised into a stream of integer symbols as follows. For each block in raster order: (1) emit the DC coefficient as a difference from the previous block's DC; (2) read the 63 AC coefficients in zig-zag order, which places low frequencies first and zeros last, and emit them up to the last non-zero value; (3) emit a special end-of-block symbol, EOB = 32768, which lies outside the range of any coefficient of an 8-bit image and thus cannot collide with data. The decoder inverts the process: it accumulates the DC differences, reads values until EOB, fills the rest of the block with zeros and undoes the zig-zag permutation.

6.5 Huffman Coding

6.5.1 Frequency calculation

A hash table (`collections.Counter`) counts the occurrences of each distinct symbol in the stream in O(N) time.

6.5.2 Tree construction with a min-heap

Each distinct symbol becomes a leaf node keyed by (frequency, insertion order). The two nodes with the smallest keys are repeatedly removed from the heap and joined under a new internal node whose frequency is the sum, and the new node is pushed back. The process stops when one node, the root, remains. The insertion-order component of the key makes the construction deterministic. The pseudocode is:

```
BUILD-HUFFMAN(freq)
  heap <- empty min-heap
  for each (symbol, f) in sorted(freq):  push (f, order, Leaf(symbol))
  while |heap| > 1:
```

```
    (f1, left)  <- pop-min(heap)
    (f2, right) <- pop-min(heap)
    push (f1 + f2, new order, Node(left, right))
  return the only node left in heap
```

6.5.3 Code generation

A depth-first traversal labels each left edge 0 and each right edge 1; the path from the root to a leaf is that symbol's code word. The traversal uses an explicit stack, because the tree can be as deep as n − 1 for skewed distributions and Python's recursion limit would otherwise be a risk. If only one distinct symbol exists, it receives the code word 0 so that the stream is not empty. Because the code is prefix-free, no code word is the prefix of another and decoding is unambiguous.

6.5.4 Encoding and decoding

Encoding concatenates the code words of all symbols and packs the resulting bit string into bytes with `numpy.packbits`; the number of valid bits is stored in the header. Decoding unpacks the bytes, truncates to the stored bit count and walks the tree from the root, taking the left child on a 0 bit and the right child on a 1 bit, emitting a symbol and returning to the root whenever a leaf is reached. A stream that ends inside a code word is reported as corrupt.

6.6 Reconstruction

Decompression executes: parse the header and table; rebuild the Huffman tree; decode the symbol stream; rebuild the integer blocks; dequantise; apply the inverse DCT; add 128, round and clip to [0, 255]; merge the blocks and crop the padding. The result is the grayscale reconstruction.

7. Implementation

7.1 Environment and Libraries

The tool is written in Python 3.11 or newer (developed and tested on Python 3.12). NumPy provides array handling and bit packing, SciPy the DCT, OpenCV image I/O and colour conversion, and Matplotlib the evaluation figures. The standard library supplies `heapq` for the priority queue and `struct` for the file container. The dependencies are listed in `requirements.txt`.

7.2 Project Structure

```
ImageCompressionTool/
  src/   image_processor.py  transform_block.py  huffman_node.py
huffman_tree.py
         encoder.py  decoder.py  compression_manager.py
decompression_manager.py
         main.py  utils.py  evaluation.py
  input/ sample_image.jpg        output/  compressed.bin, reconstructed.png,
figures
  docs/  Design_Document.md  Complexity_Analysis.md  Algorithm_Explanation.md
  tests/ test_roundtrip.py       scripts/ sample-image and diagram generators
  requirements.txt  setup.cfg  README.md  LICENSE
```

7.3 Key Code Excerpts

The heart of the greedy algorithm is the tree-building loop in `HuffmanTree.build`:

```
heap = []
for order, (symbol, freq) in enumerate(sorted(frequencies.items())):
    heapq.heappush(heap, (freq, order, HuffmanNode(freq, symbol)))
counter = len(heap)
while len(heap) > 1:
    f1, _, left = heapq.heappop(heap)
    f2, _, right = heapq.heappop(heap)
    parent = HuffmanNode(f1 + f2, None, left, right)
    heapq.heappush(heap, (parent.frequency, counter, parent))
    counter += 1
self.root = heap[0][2]
```

The symbol stream is produced by `Encoder.blocks_to_symbols`:

```
zz = quantized.reshape(n_blocks, -1)[:, ZIGZAG]
previous_dc = 0
for row in zz:
    dc = int(row[0]); symbols.append(dc - previous_dc); previous_dc = dc
    nonzero = np.flatnonzero(row[1:])
    if nonzero.size:
        symbols.extend(row[1:nonzero[-1] + 2].tolist())
    symbols.append(EOB_SYMBOL)
```

The block transform is applied to all blocks at once by operating on the last two axes:

```
shifted = blocks.astype(np.float64) - 128.0
coeffs = dctn(shifted, type=2, norm="ortho", axes=(-2, -1))
quantized = np.rint(coeffs / self.quant_table).astype(np.int32)
```

7.4 Command-Line Interface

```
python src/main.py compress   input/sample_image.jpg -q 50 -o
output/compressed.bin
python src/main.py decompress output/compressed.bin --original
input/sample_image.jpg
python src/main.py evaluate   input/sample_image.jpg
```

7.5 Testing

Eight unit tests cover: prefix-freeness and decode round trip of Huffman codes; the single-symbol tree; invertibility of the DCT (exact after rounding for random 8-bit blocks); division and merging of blocks for an image whose size is not a multiple of eight; full-pipeline round trips for an odd-sized random image and for a constant image (which must be reproduced exactly); rejection of a corrupt file; and a file-based round trip starting from a colour PNG. All tests pass.

8. Experimental Results

8.1 Experimental Setup

All experiments were run with the code described above on a single development machine under Python 3.12 with NumPy 2.4, SciPy 1.17 and OpenCV 4.13. The compression ratio is computed with respect to the raw size of the grayscale image (H·W bytes), so that results are

independent of the container format of the input file. Sizes include the 26-byte header and the frequency table. Photo 1 and Photo 2 are sample photographs distributed with Matplotlib and scikit-learn respectively; the synthetic image is generated by `scripts/generate_sample_image.py`. The synthetic image is much smoother than a typical photograph and therefore compresses unrealistically well, so the photographs should be taken as more representative.

8.2 Compression Ratio and Quality

Table 4: Results on the synthetic sample image (512×512, 262,144 raw bytes)

Quality	Bytes	Ratio	bpp	MSE	PSNR (dB)
10	3,382	77.51:1	0.10	27.95	33.67
25	6,407	40.92:1	0.20	13.49	36.83
50	9,859	26.59:1	0.30	8.63	38.77
75	17,656	14.85:1	0.54	5.40	40.81
90	50,085	5.23:1	1.53	2.91	43.49

Table 5: Results on Photo 1 (512×600, 307,200 raw bytes)

Quality	Bytes	Ratio	bpp	MSE	PSNR (dB)
10	9,426	32.59:1	0.25	90.50	28.56
25	17,598	17.46:1	0.46	46.58	31.45
50	28,538	10.76:1	0.74	27.49	33.74
75	56,977	5.39:1	1.48	3.44	42.77
90	67,486	4.55:1	1.76	0.33	52.91

Table 6: Results on Photo 2 (640×427, 273,280 raw bytes)

Quality	Bytes	Ratio	bpp	MSE	PSNR (dB)
10	11,501	23.76:1	0.34	292.27	23.47
25	23,678	11.54:1	0.69	174.03	25.72
50	39,429	6.93:1	1.15	97.72	28.23
75	59,677	4.58:1	1.75	40.56	32.05
90	93,731	2.92:1	2.74	8.40	38.89

As expected, higher quality lowers distortion and raises the file size (Figure 4). The ratios for the photographs at quality 50 (10.8:1 and 6.9:1) are well below the synthetic image’s 26.6:1, because natural images contain more high-frequency detail and therefore more non-zero coefficients. The very steep rise in PSNR of Photo 1 between quality 50 and 90 may be related to the fact that this image was itself stored as a JPEG, so that its coefficients lie close to a quantisation grid; this was not investigated further.

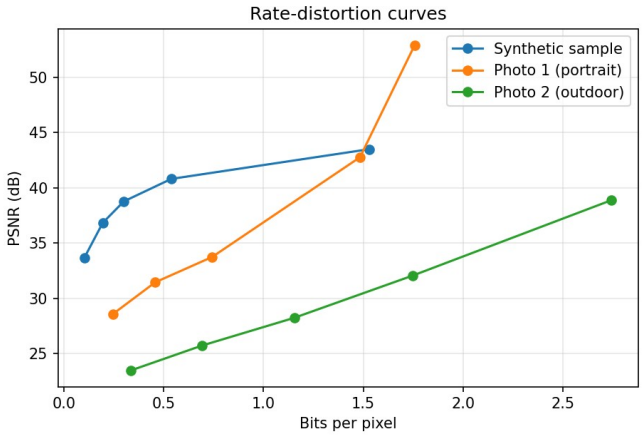


Figure 4: Rate–distortion curves for the three test images



Figure 5: Original Photo 1 and reconstructions at quality 10, 50 and 90 (compression ratio and PSNR shown above each image)

Visually (Figure 5), quality 90 is indistinguishable from the original and quality 50 shows only minor softening. At quality 10 the characteristic 8×8 blocking artefacts are visible in the smooth background, a known limitation of block transforms at low bit rates.

8.3 Symbol Statistics

Table 7: Internal statistics on the synthetic image

Quality	Symbols	Distinct	Zero coefficients	Avg. code (bits)	Payload (B)	Table (B)
10	11,205	27	98.1%	2.24	3,140	216
50	25,068	107	95.2%	2.87	8,977	856
90	163,979	407	76.0%	2.28	46,803	3,256

At quality 10, 98.1% of all quantised coefficients are zero, which the zig-zag scan and end-of-block marker reduce to a short symbol stream (11,205 symbols for 4,096 blocks). The frequency table is a small but non-negligible overhead: 216 bytes (6% of the file) at quality 10 and 3,256 bytes (6.5%) at quality 90. Storing canonical code lengths would reduce this cost.

8.4 Ablation: Contribution of Each Design Element

To quantify the contribution of each stage, the payload size at quality 50 (excluding header and table) was measured for four symbol models. Because Huffman coding is applied to individual

values, the order in which values are emitted does not change the histogram, and raster order and zig-zag order give identical sizes unless an end-of-block marker is used.

Table 8: Huffman payload size (bytes) at quality 50 for different symbol models

Symbol model	Photo 1	Photo 2	Synthetic
Raw pixels, no transform (whole file)	273,157	265,971	234,189
All 64 quantised values per block	56,354	60,922	39,482
Zig-zag + EOB, raw DC	29,718	41,949	11,415
Zig-zag + EOB + DC difference (final)	27,096	38,435	8,977

- **Transform and quantisation** reduce the size from roughly 234,000–273,000 bytes (1.03:1 to 1.12:1, essentially no compression) to 39,000–61,000 bytes.
- **Zig-zag with end-of-block** removes a further 31% to 71% by eliminating the long runs of trailing zeros.
- **Differential DC coding** gives an additional 8% to 21%, because neighbouring blocks have similar mean intensity.

8.5 Comparison with a Reference JPEG Encoder

The same grayscale images were encoded with OpenCV's JPEG encoder (libjpeg) at the same quality factors. Because this tool uses the same DCT, level shift and quantisation tables, the reconstruction PSNR values agreed to within 0.01 dB in all tested cases, and the comparison isolates the effect of the entropy-coding and symbol-modelling stages. Table 9 compares file sizes against JPEG with optimised Huffman tables (`IMWRITE_JPEG_OPTIMIZE`), which is the fairest comparison because this tool also builds per-image tables.

Table 9: File size relative to optimised JPEG (this tool ÷ JPEG; > 1 means larger)

Quality	Photo 1	Photo 2	Synthetic
10	1.12	1.08	1.25
50	1.11	1.07	1.15
90	1.04	1.06	1.15

The files produced here are 4% to 25% larger than optimised JPEG. Against libjpeg's default (non-optimised) tables the ratio ranged from 0.72 to 1.08, i.e. this tool was sometimes smaller. The remaining gap to optimised JPEG is attributable to the absence of joint (run, size) symbols, to the explicit frequency table, and to the stored magnitude of every coefficient, while JPEG codes a magnitude category plus raw bits. Since the transform and quantisation are identical, these results indicate that the project's entropy-coding stage is reasonable but not state of the art.

8.6 Running Time

Table 10: Running time versus image size (quality 50; synthetic image tiled)

Image size	Blocks	Compress (s)	Decompress (s)
256×256	1,024	0.008	0.005
512×512	4,096	0.022	0.022

Image size	Blocks	Compress (s)	Decompress (s)
1024×1024	16,384	0.085	0.085
2048×2048	65,536	0.373	0.282

Each doubling of the side length multiplies the time by about four, i.e. the running time is linear in the number of pixels, as predicted in Chapter 9. The absolute values are specific to the test machine; the tiled images are periodic, so only the timing, not the size, of these larger images should be interpreted.

8.7 Threats to Validity

- Only three images were used, one of them synthetic; results on larger, more diverse data sets may differ.
- PSNR is an imperfect proxy for visual quality; SSIM or subjective tests were not performed.
- Timings come from a single machine and a single run each, without repetition statistics.
- The JPEG comparison covers grayscale only and one library implementation.

9. Complexity Analysis

Let n be the number of distinct symbols, N the length of the symbol stream, P the number of pixels, B ≈ P/64 the number of blocks and b = 8 the block size. Table 11 gives the required summary; the interpretation of the variables in each row is stated below the table.

Table 11: Complexity of the main components

Component	Time Complexity	Space Complexity
Huffman Tree Build	O(n log n)	O(n)
Encoding	O(n)	O(n)
Decoding	O(n)	O(n)
Block Processing	O(b)	O(b)
DCT Transform	$O(b^2)$	$O(b^2)$

In the rows for encoding and decoding, “n” denotes the length of the data being processed (N above); in “Block Processing”, “b” is the number of blocks (B); and in “DCT Transform”, b^2 is the number of samples in one block. More precisely, for this implementation:

- **Tree build:** n insertions and n − 1 merges, each O(log n), give O(n log n) time and O(n) space. Code generation is O(n) in the number of nodes.
- **Encoding:** O(N) symbol look-ups plus $O(N \cdot \bar{L})$ bit-string work, where $\bar{L} \approx 2–3$ bits is the average code length; O(N) space for the output.
- **Decoding:** one tree step per bit, $O(N \cdot \bar{L}) = O(N)$ time, O(N) space.
- **Block processing:** splitting and merging blocks are single passes, O(P) = O(B) for fixed b.

- **DCT per block:** a separable DCT written as two matrix products costs $O(b^3)$ per block; a fast algorithm costs $O(b^2 \log b)$. Since b is the constant 8, the cost per block is O(1) and the cost for the whole image is O(P).
- **Whole pipeline:** $O(P + n \log n)$. Because n is at most a few hundred for realistic quality settings and P is in the hundreds of thousands or millions, the pipeline is effectively linear in the image size, which matches the measurements in Section 8.6.

10. Advantages

- **Transparent and modular:** each stage is a separate, testable class, which makes the system suitable for teaching and experimentation.
- **Good compression for its simplicity:** ratios of 7:1 to 11:1 on natural photographs at PSNR above 28 dB, within roughly 4–25% of optimised JPEG at essentially identical distortion.
- **Tunable quality:** one integer parameter spans near-lossless to heavily compressed output.
- **Optimal prefix code per image:** the Huffman table adapts to the statistics of each image.
- **Linear running time** and trivially parallel block transform.
- **Robust:** self-describing format, input validation and explicit error messages; no unsafe deserialisation.
- **Reproducible:** tests, scripts and documentation are included.

11. Limitations

- **Lossy:** quantisation discards information; exact recovery is not possible except approximately at quality 100.
- **Grayscale only:** colour images are converted to luminance and the colour information is lost.
- **Blocking artefacts** appear at low quality settings because blocks are coded independently.
- **Symbol model:** the absence of run-length symbols and context modelling leaves 4–25% on the table compared with optimised JPEG.
- **Integer code lengths:** Huffman coding cannot go below one bit per symbol and cannot reach the entropy exactly, which matters when most symbols are highly probable.
- **Table overhead:** storing frequencies costs 8 bytes per distinct symbol, about 6% of the file in the measured cases.
- **Pure-Python entropy stage:** tree walking and bit-string construction are slower than a compiled implementation for very large images.
- **Limited evaluation:** three images, PSNR only.

12. Future Scope

- **Colour support:** convert to YCbCr, subsample chroma and code the three planes separately.
- **Better symbol model:** joint (zero-run, size) symbols as in baseline JPEG, which is expected to close much of the gap to optimised JPEG.
- **Canonical Huffman codes:** store code lengths instead of frequencies to shrink the table.
- **Adaptive or arithmetic coding:** approach the entropy more closely.
- **Deblocking filter and adaptive quantisation** to improve visual quality at low rates.
- **Variable block size** (4×4 to 32×32), chosen per region.
- **Parallel processing** of blocks with multiprocessing or a compiled extension.
- **Additional metrics:** SSIM and perceptual measures, and evaluation on standard data sets such as Kodak.
- **Graphical interface** for interactive quality selection and side-by-side viewing.

13. Conclusion

This project delivered a complete, tested and documented image compression tool that combines block-wise DCT processing with Huffman coding. The design follows the classical transform-coding architecture: independent 8×8 blocks (divide and conquer), orthonormal DCT, quality-controlled quantisation, differential DC coding with zig-zag scanning and end-of-block markers, and a greedy Huffman code built with a min-heap. The decompressor reverses each step and reproduces the image with controllable distortion.

The experiments confirmed the expected behaviour. Huffman coding of raw pixels is nearly useless (1.03:1 to 1.12:1), whereas the transform, quantisation and a structured symbol stream raise the ratio to 6.9:1 and 10.8:1 on two natural photographs at quality 50, with PSNR of 28.2 dB and 33.7 dB. The end-of-block mechanism and differential DC coding each contributed measurably. Against a reference JPEG encoder the files were 4% to 25% larger at essentially identical PSNR, a gap that the proposed run-length symbol model and canonical codes should reduce. The running time grew linearly with the number of pixels, in agreement with the analysis.

Beyond the numbers, the project demonstrates how a greedy algorithm and a divide-and-conquer decomposition combine in a real application, and the modular code base provides a solid foundation for the extensions listed in Chapter 12.

References

[1] Ahmed, N., Natarajan, T., and Rao, K. R. (1974). Discrete cosine transform. *IEEE Transactions on Computers*, C-23(1), 90–93.

[2] Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2022). *Introduction to Algorithms*, 4th ed. MIT Press.

[3] Gallager, R. G. (1978). Variations on a theme by Huffman. *IEEE Transactions on Information Theory*, 24(6), 668–674.

[4] Gonzalez, R. C., and Woods, R. E. (2018). *Digital Image Processing*, 4th ed. Pearson.

[5] Huffman, D. A. (1952). A method for the construction of minimum-redundancy codes. *Proceedings of the IRE*, 40(9), 1098–1101.

[6] Loeffler, C., Ligtenberg, A., and Moschytz, G. S. (1989). Practical fast 1-D DCT algorithms with 11 multiplications. In *Proc. IEEE ICASSP*, 988–991.

[7] Pennebaker, W. B., and Mitchell, J. L. (1993). *JPEG: Still Image Data Compression Standard*. Van Nostrand Reinhold.

[8] Rao, K. R., and Yip, P. (1990). *Discrete Cosine Transform: Algorithms, Advantages, Applications*. Academic Press.

[9] Schwartz, E. S., and Kallick, B. (1964). Generating a canonical prefix encoding. *Communications of the ACM*, 7(3), 166–169.

[10] Shannon, C. E. (1948). A mathematical theory of communication. *Bell System Technical Journal*, 27, 379–423 and 623–656.

[11] Skodras, A., Christopoulos, C., and Ebrahimi, T. (2001). The JPEG 2000 still image compression standard. *IEEE Signal Processing Magazine*, 18(5), 36–58.

[12] Taubman, D. S., and Marcellin, M. W. (2002). *JPEG2000: Image Compression Fundamentals, Standards and Practice*. Kluwer Academic.

[13] Wallace, G. K. (1992). The JPEG still picture compression standard. *IEEE Transactions on Consumer Electronics*, 38(1), xviii–xxxiv.

[14] Wang, Z., Bovik, A. C., Sheikh, H. R., and Simoncelli, E. P. (2004). Image quality assessment: from error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4), 600–612.

[15] Wiegand, T., Sullivan, G. J., Bjøntegaard, G., and Luthra, A. (2003). Overview of the H.264/AVC video coding standard. *IEEE Transactions on Circuits and Systems for Video Technology*, 13(7), 560–576.

[16] Witten, I. H., Neal, R. M., and Cleary, J. G. (1987). Arithmetic coding for data compression. *Communications of the ACM*, 30(6), 520–540.

[17] Harris, C. R., et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362.

[18] Virtanen, P., et al. (2020). SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17, 261–272.

[19] Hunter, J. D. (2007). Matplotlib: a 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95.

[20] Bradski, G. (2000). The OpenCV library. *Dr. Dobb's Journal of Software Tools*.